

第七章 数据库设计和E-R模型

7.1 设计过程概览

7.1.1 设计阶段

- 完整刻画用户的数据需求
- 选择数据模型
- **概念设计阶段**
- 完善概念模式，指明功能需求（**功能需求规格说明**）
- **逻辑设计阶段、物理设计阶段**

7.1.2 设计选择

实体指示所有可以明确的个体

设计数据库模式时，必须避免两个主要缺陷：

- **冗余**：不好的设计可能会导致重复信息
最大的问题是，当对一条消息进行更新时，这条消息的拷贝可能会与其不一致
- **不完整**：某些方面可能难以甚至无法建模

7.2 实体-联系模型

E-R数据模型采用三个基本概念：实体集、联系集、属性

7.2.1 实体集

实体是现实世界中可区别于所有其他对象的一个“事物”或“对象”

实体集是相同类型具有相同性质（属性）的一个实体集合

实体通过一组**属性**来表示

每个实体的每个属性都有一个**值**

7.2.2 联系集

联系是指多个实体间的相互关联

联系集是相同类型联系的集合

实体集 $E_1, E_2, \dots, E_n$ 参与联系集R

实体在联系中扮演的功能称为实体的**角色**

联系也可以具有**描述性属性**

参与联系集的实体集的数目称为联系集的**度**（二元联系集的度为2，三元联系集的度为3）

7.2.3 属性

域（值集）：每个属性可取值的集合

属性类型：

- **简单和复合属性**：
复合属性可以划分为更小的部分（其它属性）



- **单值和多值属性：**
 单值属性：一个属性只有单独的一个值
 多值属性：一个属性可能对应于一组值
- **派生属性：**
 这类属性的值可以从别的相关属性或实体派生出来

7.3 约束

7.3.1 映射基数

映射基数：表示一个实体通过一个联系集能关联的实体的个数

- 一对一
- 一对多
- 多对一
- 多对多

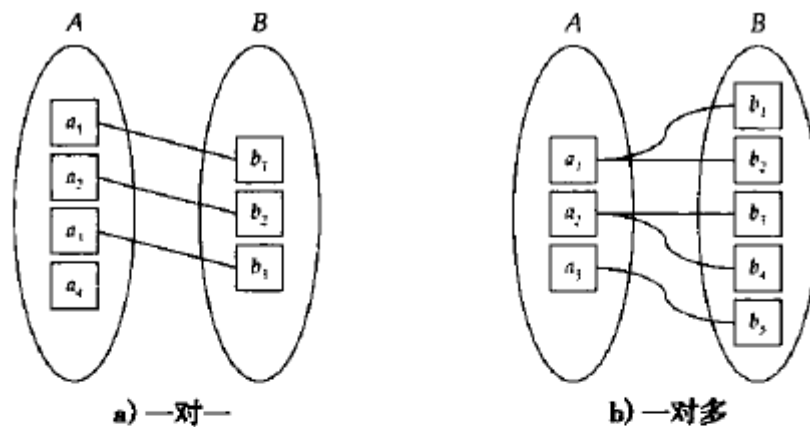


图 7-5 映射基数

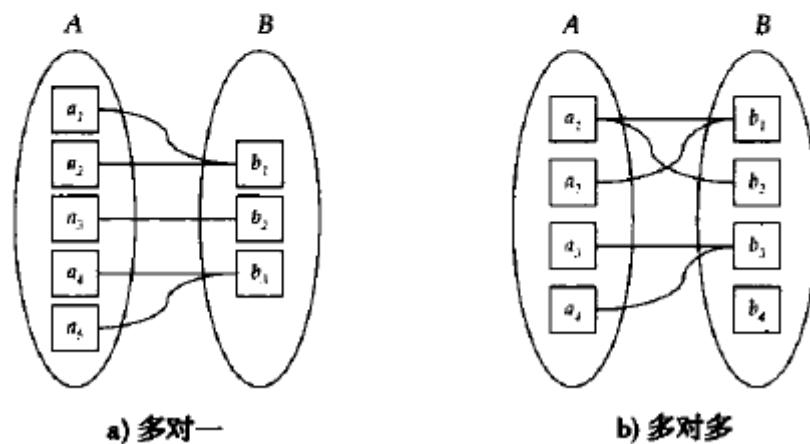


图 7-6 映射基数

7.3.2 参与约束

如果实体集E中的每个实体都参与到联系集R的至少一个联系中，则称E在R中的参与是**全部的**

7.3.3 码

一个实体的属性的值必须可以唯一标识该物体

联系集的主码结构依赖于联系集的映射基数

7.5 实体-联系图

E-R图可以图形化标识数据库的全局逻辑结构

7.5.1 基本结构

E-R 图包括如下几个主要构件：

- 分成两部分的矩形代表实体集。本书中有阴影的第一部分包含实体集的名字，第二部分包含实体集中所有属性的名字。
- 菱形代表联系集。
- 未分割的矩形代表联系集的属性。构成主码的属性以下划线标明。
- 线段将实体集连接到联系集。
- 虚线将联系集属性连接到联系集。
- 双线显示实体在联系集中的参与度。
- 双菱形代表连接到弱实体集的标志性联系集（我们将在 7.5.6 节讲述标志性联系集和弱实体集）。



图 7-7 对应于教师和学生的 E-R 图

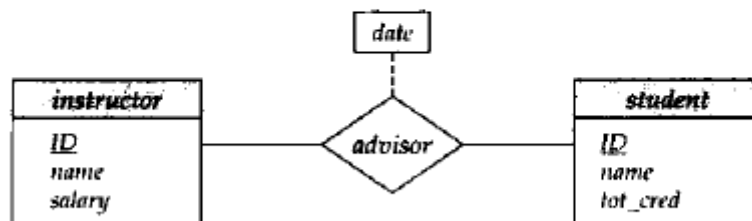


图 7-8 联系集上附带了一个属性的 E-R 图

7.5.2 映射基数

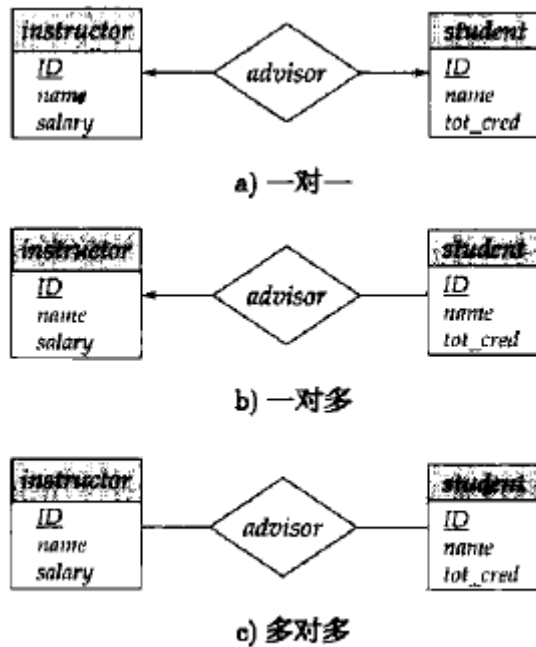


图 7-9 联系

7.5.3 复杂的属性

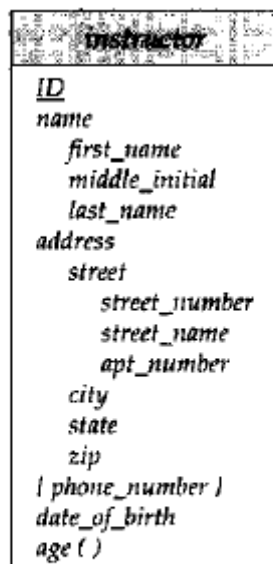


图 7-11 包含复合、多值和派生属性的 E-R 图

7.5.6 弱实体集

弱实体集：没有足够的属性以形成主码的实体集

强实体集：有主码的实体集

弱实体集必须与另一个称作**标识或属主实体集**的实体集关联才有意义

弱实体集**存在依赖**于标识实体集；标识实体集**拥有**它所标识的弱实体集

弱实体集与其标识实体集相连的联系称为**标识性联系**

弱实体集的**分辨符**可以区分其中的实体

在 E-R 图中，弱实体集和强实体集类似，以矩形表示，但是有两点主要的区别：

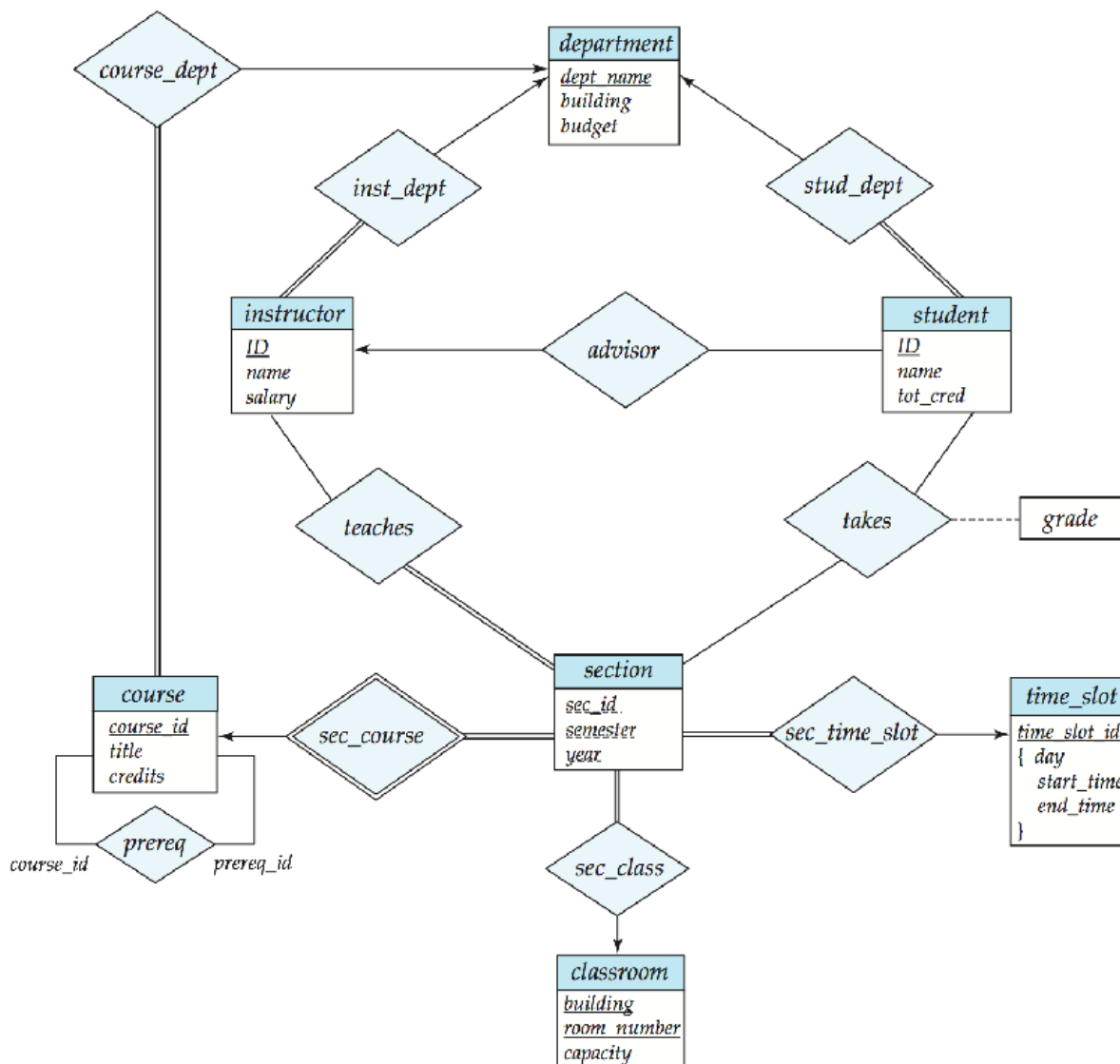
- 弱实体集的分标识符以虚下划线标明，而不是实线。
- 关联弱实体集和标识性强实体集的联系集以双菱形表示。

在图 7-14 中，弱实体集 *section* 通过联系集 *sec_course* 依赖于强实体集 *course*。



图 7-14 包含弱实体集的 E-R 图

7.5.7 大学的E-R图



7.7 实体-联系设计问题

7.7.1 用实体集还是用属性

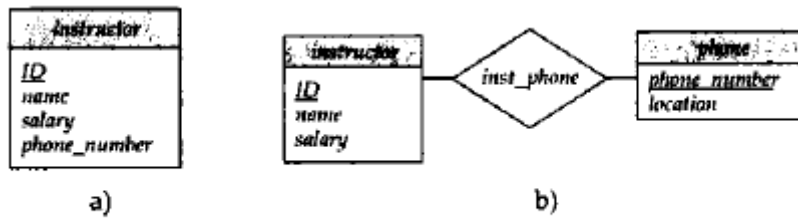


图 7-17 给 *instructor* 实体集增加 *phone* 的两种方法

常见错误：用一个实体集的主码作为另一个实体集的属性，而不是用联系

7.7.2 用实体集还是用联系集

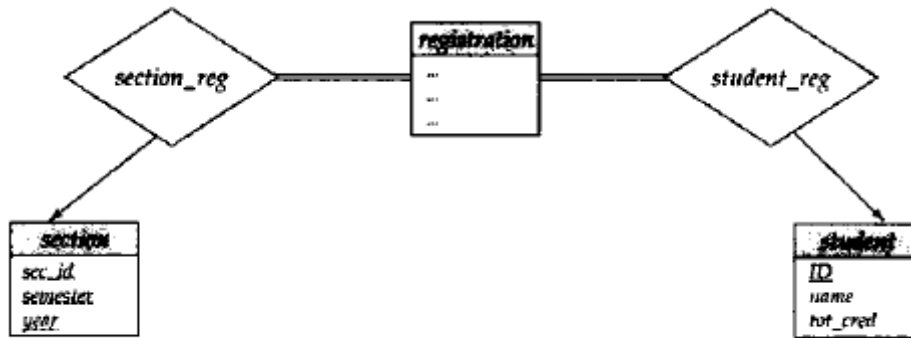


图 7-18 用 *registration* 和两个联系集替代 *takes*

原则：当描述发生在实体间的行为时采用联系集

7.7.3 二元还是n元联系集

数据库中的联系通常都是二元的，一些看来非二元的联系也可以用多个二元联系表示

7.7.4 联系属性的布局

一对一或一对多联系集的属性可以放到一个参与该联系的实体集中，而不是放到实体集中

7.8 扩展的E-R特性

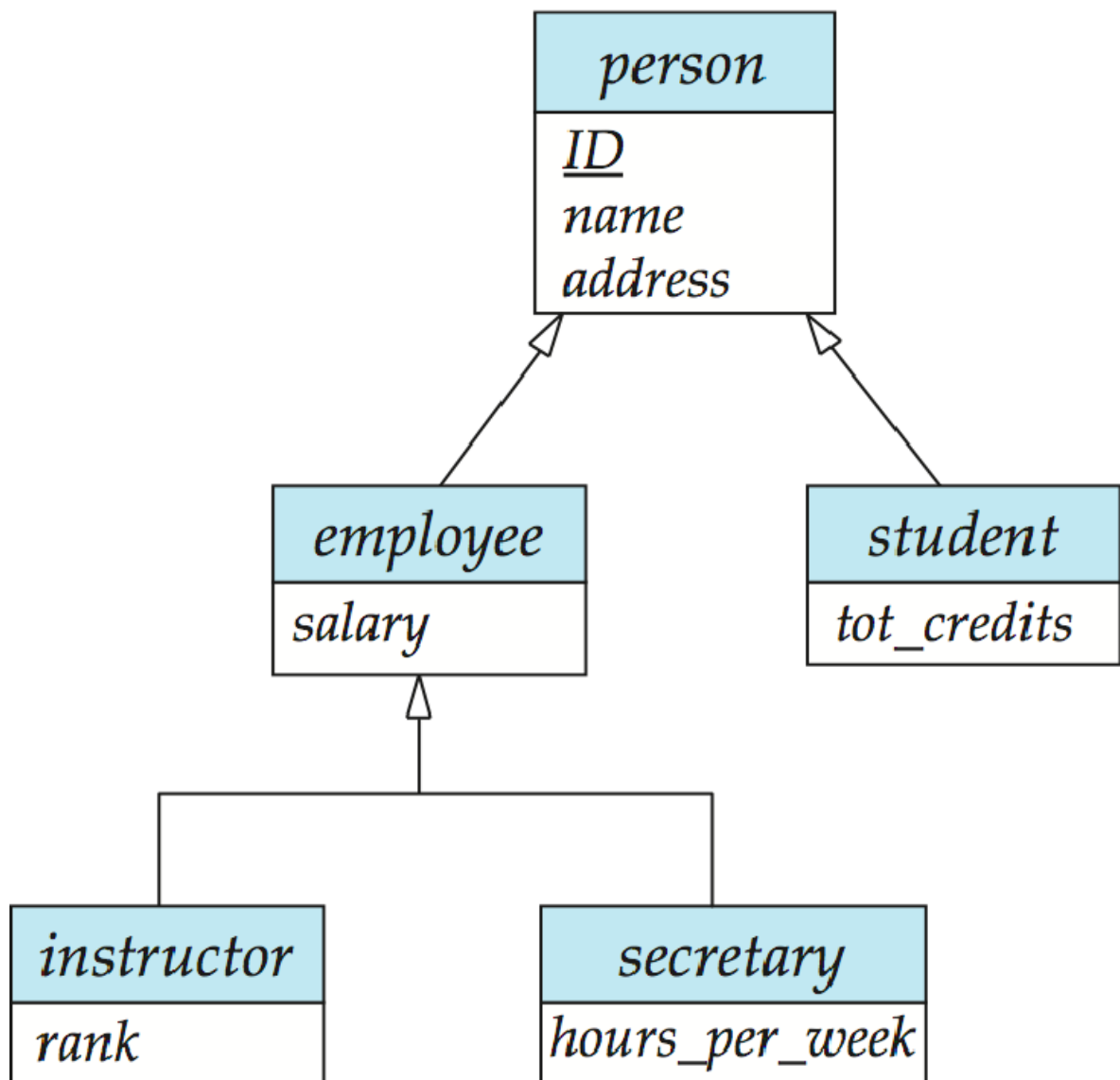
扩展E-R特性：特化、概化、高层和低层实体集属性继承、聚集

7.8.1 特化

在实体集内部进行分组的过程称为**特化** (e.g. person->employee, student)

重叠特化：一个实体集可能属于多个特化实体集

不相交特化：一个实体集属于至多一个特化实体集



7.8.2 概化

实体集间包含相同的属性可以通过**概化**来表达，概化是高层实体集与一个或多个实体集间的包含关系

高层与低层实体集也可以分别称作**超类**和**子类**

概化本质上是**特化**的逆过程

7.8.3 属性继承

由特化和概化所产生的高层和低层实体的一个重要特性是**属性继承**

7.8.4 概化上的约束

判定哪些实体能成为给定低层实体集的成员的条件：

- **条件定义的**：成员资格的确定基于实体是否满足一个显式的条件或谓词
- **用户定义的**：低层实体集由用户将实体指派给某个实体集

一个概化中一个实体是否可以属于多个低层实体集：

- **不相交**：一个实体至多属于一个低层实体集
- **重叠**：同一个实体可以同时属于多个低层实体集

对概化的**完全性约束**：

- **全部概化或特化**：每个高层实体必须属于一个低层实体集
- **部分概化或特化**：一些高层实体不属于任何低层实体集

7.8.5 聚集

E-R模型的一个局限性在于它不能表达联系间的联系

聚集是一种抽象，通过这种抽象，联系被视为高层实体

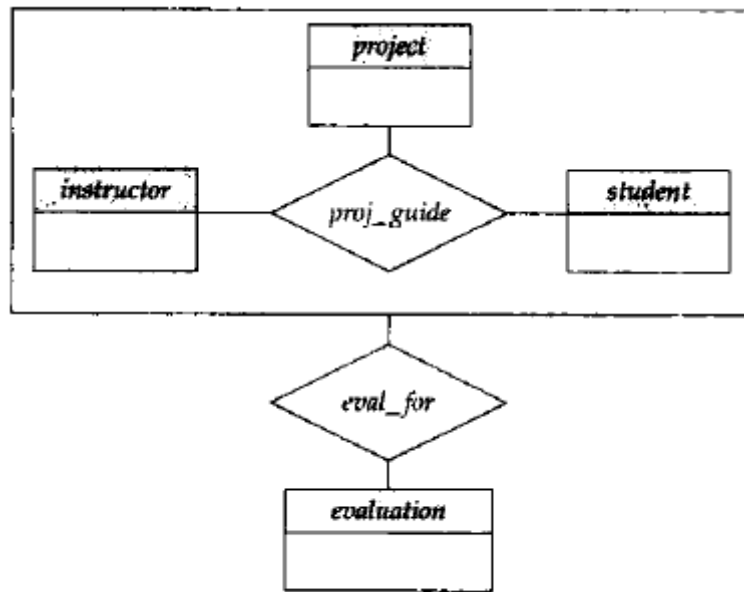


图 7-23 包含聚集的 E-R 图

7.11 总结

见P176

第八章 关系数据库设计

8.1 好的关系设计的特点

我们要如何发现信息重复，并把他分表呢？

如果存在模式(dept_name,budget) 则dept_name可以作为主码。这条规则被称为**函数依赖**

$$dept_name \rightarrow budget$$

由于dept_name不能是inst_dept的主码，因此budget可能会重复。可以拆出一张dept_name为主码的表。

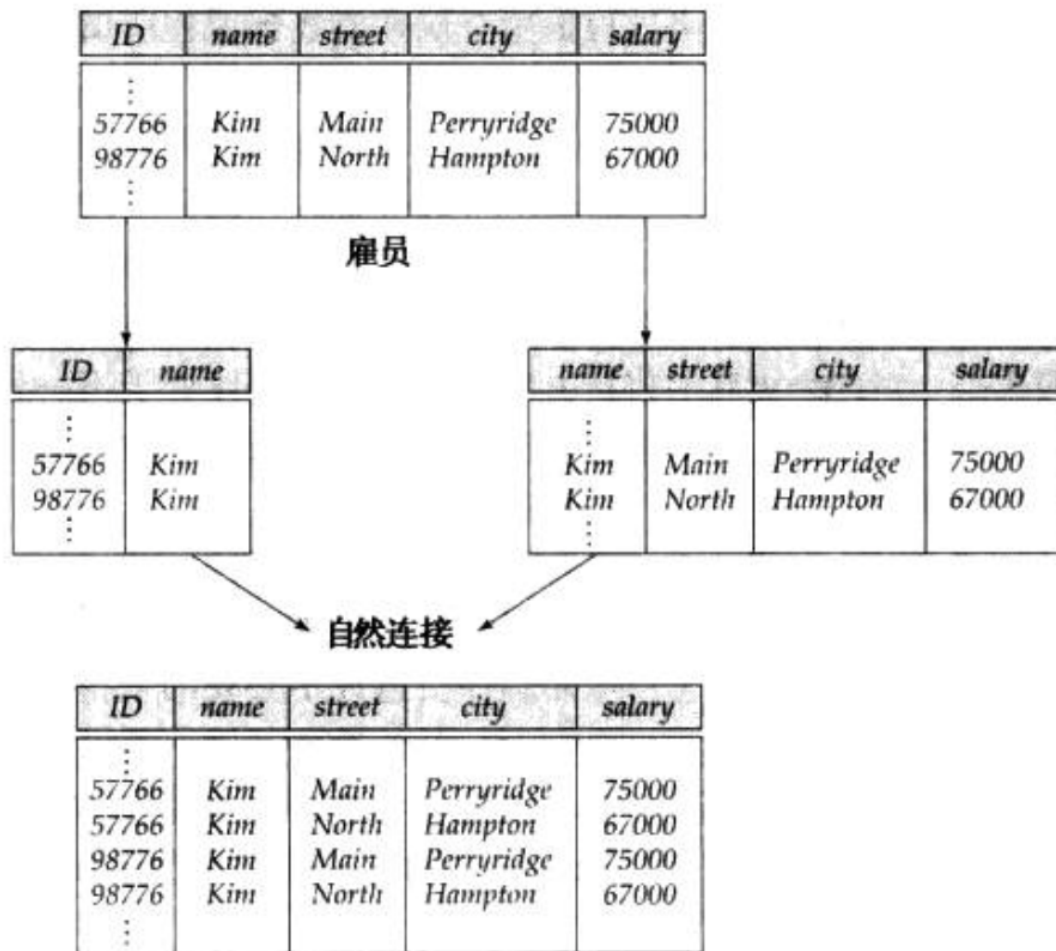
我们将下面的employee表分解成两个模式：

employee (ID, name, street, city, salary)

employee1 (ID, name)

employee2 (name, street, city, salary)

但因为name一般不是唯一的，如果用来做主码，在连接两张表的时候就会产生错误。这就是**有损分解**，反之是**无损分解**



8.2 原子域和第一范式

一个域是**原子的**，如果该域的元素被认为是不可分的单元

关系模式R属于**第一范式 (1NF)**，如果R的所有属性的域都是原子的

8.3 使用函数依赖进行分解

属性集： α

关系模式： $r(R)$ 该模式是关系r的，R是属性集

超码：K

一个关系模式是一个属性集，但是并非所有的属性集都是模式

8.3.1 码和函数依赖

一个关系的满足所有这种现实世界约束的实例，称为关系的**合法实例**

R的子集K是r(R)的**超码**的条件：在关系r(R)的任一合法实例中，对于r的实例中的所有元组对 t_1 和 t_2 总满足，若 $t_1 \neq t_2$ ，则 $t_1[K] \neq t_2[K]$ ，

考虑一个关系模式r(R)，令 $\alpha \subseteq R$ 且 $\beta \subseteq R$ ：

- 满足**函数依赖** $\alpha \rightarrow \beta$ 的条件是：对实例中所有元组对 t_1 和 t_2 ，若 $t_1[\alpha] = t_2[\alpha]$ ，则 $t_1[\beta] = t_2[\beta]$ (单射)
- 如果在r(R)的每个合法实例中都满足函数依赖 $\alpha \rightarrow \beta$ ，则函数依赖在模式r(R)上**成立**

两种方式使用函数依赖：

- 判定关系的实例是否满足给定函数依赖集F

- 说明合法关系集上的约束

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_3	c_2	d_3
a_3	b_3	c_2	d_4

如上图例子，注意到 $A \rightarrow C$ 是满足的 但是 $D \rightarrow A$ 是不满足的

一个关系实例可能满足某些函数依赖，但这些函数依赖并不一定在关系的模式上成立。比如下面 $room_number \rightarrow capacity$ 是满足的，但我们相信不同建筑里两个相同编号的教室，可能大小不同。不过我们会期望 $building, room_number \rightarrow capacity$ 在 *classroom* 模式上成立

<i>building</i>	<i>room_number</i>	<i>capacity</i>
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

平凡的：有些函数依赖称为**平凡的**，因为它们在所有关系中都满足，比如 $A \rightarrow A$ ，同样的 $AB \rightarrow A$ 也满足

如果 $\beta \subseteq \alpha$ ，则形如 $\alpha \rightarrow \beta$ 的函数依赖是**平凡的**

F^+ 符号表示F集合的**闭包**，表示F集合推导出的所有函数依赖的集合，比如说：

$A \rightarrow B + B \rightarrow C = A \rightarrow C$

8.3.2 Boyce-Codd 范式 (BCNF)

它消除所有基于函数依赖能够发现的冗余。

具有函数依赖集F的关系模式R属于BCNF的条件：

对 F^+ 中所有形如 $\alpha \rightarrow \beta$ 的函数依赖（其中 $\alpha \subseteq R$ 且 $\beta \subseteq R$ ），下面至少有一项成立：

- $\alpha \rightarrow \beta$ 是平凡的函数依赖（即 $\beta \subseteq \alpha$ ）
- α 是模式R的一个超码

Example schema not in BCNF:

instr_dept (ID, name, salary, dept_name, building, budget)

because $dept_name \rightarrow building, budget$
holds on *instr_dept*, but *dept_name* is not a superkey

一个数据库属于BCNF的条件是：构成该设计的关系模式集中的每个模式都属于BCNF

非BCNF模式转换为BCNF的分解：

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

- $(\alpha \cup \beta) = (\text{dept_name}, \text{building}, \text{budget})$
- $(R - (\beta - \alpha)) = (ID, \text{name}, \text{salary}, \text{dept_name})$

8.3.4 第三范式

具有函数依赖集F的关系模式R属于**第三范式**的条件是：

对于 F^+ 中所有形如 $\alpha \rightarrow \beta$ 的函数依赖（其中 $\alpha \subseteq R$ 且 $\beta \subseteq R$ ），以下至少一项成立：

- $\alpha \rightarrow \beta$ 是一个平凡的函数依赖
- α 是R的一个超码
- $\beta - \alpha$ 中的每个属性A都包含于R的一个候选码中（可以包含于不同的候选码）

8.4 函数依赖理论

8.4.1 函数依赖集的闭包

如果关系模式r(R)的每一个满足F的实例也满足f，则R上的函数依赖f被r上的函数依赖集F**逻辑蕴涵**（ $A \rightarrow B, B \rightarrow C$ 逻辑蕴涵 $A \rightarrow C$ ）

F的**闭包**是被F逻辑蕴涵的所有函数依赖的集合，记作 F^+

Armstrong公理：正确有效的，完备的

- **自反律**：若 α 为一属性集且 $\beta \subseteq \alpha$ ，则 $\alpha \rightarrow \beta$ 成立
- **增补律**：若 $\alpha \rightarrow \beta$ 成立且 γ 为一属性集，则 $\gamma\alpha \rightarrow \gamma\beta$
- **传递律**：若 $\alpha \rightarrow \beta$ 和 $\beta \rightarrow \gamma$ 成立，则 $\alpha \rightarrow \gamma$ 成立
- **合并律 (union rule)**。若 $\alpha \rightarrow \beta$ 和 $\alpha \rightarrow \gamma$ 成立，则 $\alpha \rightarrow \beta\gamma$ 成立。
- **分解律 (decomposition)**。若 $\alpha \rightarrow \beta\gamma$ 成立，则 $\alpha \rightarrow \beta$ 和 $\alpha \rightarrow \gamma$ 成立。
- **伪传递律 (pseudotransitivity rule)**。若 $\alpha \rightarrow \beta$ 和 $\gamma\beta \rightarrow \delta$ 成立，则 $\alpha\gamma \rightarrow \delta$ 成立。

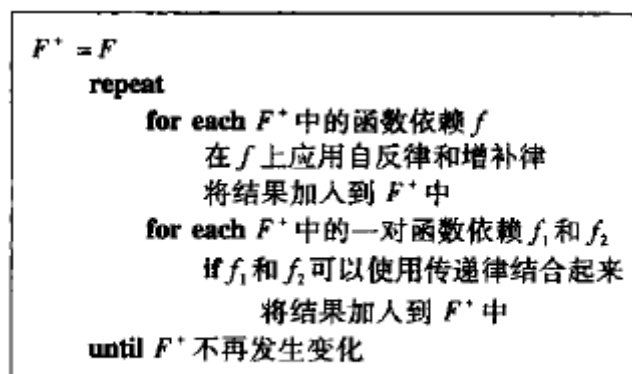


图 8-7 计算 F^+ 的过程

8.4.2 属性集的闭包

如果 $\alpha \rightarrow B$ ，我们称属性B被 α **属性确定**

令 α 为一个属性集，F下被 α 确定的所有属性的集合称为F下 α 的**就闭包**，记为 α^+

```

result :=  $\alpha$ ;
repeat
  for each 函数依赖  $\beta \rightarrow \gamma$  in  $F$  do
    begin
      if  $\beta \subseteq \text{result}$  then  $\text{result} := \text{result} \cup \gamma$ ;
    end
until ( $\text{result}$  不变)

```

图 8-8 计算 F 下 α 的闭包 α^+ 的算法

■ $R = (A, B, C, G, H, I)$

■ $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$

■ $(AG)^+$

1. $\text{result} = AG$

2. $\text{result} = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)

3. $\text{result} = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)

4. $\text{result} = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

属性闭包算法有多种用途：

- 判断 α 是否为超码：计算 α^+ ，检查 α^+ 是否包含 R 中的所有属性
- 检查函数依赖 $\alpha \rightarrow \beta$ 是否成立：通过检查是否 $\beta \subseteq \alpha^+$ （是否属于 F^+ ），即计算 α^+ 看是否包含 β
- 计算 F^+ 方法：对任意的 $\gamma \subseteq R$ ，我们找出闭包 γ^+ ；对任意的 $S \subseteq \gamma^+$ ，我们输出一个函数依赖 $\gamma \rightarrow S$

Is AG a candidate key?

1. Is AG a super key?

1. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$

2. Is any subset of AG a superkey?

1. Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$

2. Does $G \rightarrow R$? == Is $(G)^+ \supseteq R$

8.4.3 正则覆盖

如果去除函数依赖中的一个属性不改变该函数依赖集的闭包，则称该属性是**无关的**

无关属性的形式化定义如下：考虑函数依赖集 F 及 F 中的函数依赖 $\alpha \rightarrow \beta$

- 如果 $A \in \alpha$ 并且 F 逻辑蕴含 $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$, 则属性 A 在 α 中是无关的。
- 如果 $A \in \beta$ 并且函数依赖集 $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ 逻辑蕴含 F , 则属性 A 在 β 中是无关的。

有效检验一个属性是否无关的方法, 考虑 $\alpha \rightarrow \beta$ 中的一个属性 A :

- 如果 $A \in \beta$, 为了检验 A 是否是无关的, 考虑集合

$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$$
并检验 $\alpha \rightarrow A$ 是否能够由 F' 推出。为此, 计算 F' 下的 α^+ (α 的闭包); 如果 α^+ 包含 A , 则 A 在 β 中是无关的。
- 如果 $A \in \alpha$, 为了检验 A 是否是无关的, 令 $\gamma = \alpha - \{A\}$, 并且检查 $\gamma \rightarrow \beta$ 是否可以由 F 推出。为此, 计算在 F 下的 γ^+ (γ 的闭包); 如果 γ^+ 包含 β 中的所有属性, 则 A 在 α 中是无关的。

example:

例如, 假定 F 包含 $AB \rightarrow CD$ 、 $A \rightarrow E$ 和 $E \rightarrow C$ 。为检验 C 在 $AB \rightarrow CD$ 中是否是无关的, 我们计算 $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$ 下 AB 的属性闭包。闭包为 $ABCDE$, 包含 CD , 所以我们推断出 C 是无关的。

F 的正则覆盖 F_c 是一个依赖集, 使得 F 逻辑蕴含 F_c 中的所有依赖, 并且 F_c 逻辑蕴含 F 中的所有依赖, 且 F_c 必须具有如下性质:

- F_c 中任何函数依赖都不含无关属性
- F_c 中函数依赖的左半部分都是唯一的, 即 F_c 中不存在两个依赖 $\alpha_1 \rightarrow \beta_1$ 和 $\alpha_2 \rightarrow \beta_2$, 满足 $\alpha_1 = \alpha_2$

```

F_c = F
repeat
    使用合并律将 F_c 中所有形如  $\alpha_1 \rightarrow \beta_1$  和  $\alpha_1 \rightarrow \beta_2$  的依赖
    替换为  $\alpha_1 \rightarrow \beta_1\beta_2$ 
    在 F_c 中寻找一个函数依赖  $\alpha \rightarrow \beta$ , 它在  $\alpha$  或在  $\beta$  中
    具有一个无关属性
    /* 注意, 使用 F_c 而非 F 检验无关属性 */
    如果找到一个无关属性, 则将它从 F_c 中的  $\alpha \rightarrow \beta$  中删除
until (F_c 不变)
  
```

图 8-9 计算正则覆盖

example:

- $R = (A, B, C)$
 $F = \{A \rightarrow BC$
 $\quad B \rightarrow C$
 $\quad A \rightarrow B$
 $\quad AB \rightarrow C\}$
- Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 - Set is now $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- A is extraneous in $AB \rightarrow C$
 - Check if the result of deleting A from $AB \rightarrow C$ is implied by the other dependencies
 - ☞ Yes: in fact, $B \rightarrow C$ is already present!
 - Set is now $\{A \rightarrow BC, B \rightarrow C\}$
- C is extraneous in $A \rightarrow BC$
 - Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - ☞ Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
 - Can use attribute closure of A in more complex cases
- The canonical cover is:
$$\begin{array}{l} A \rightarrow B \\ B \rightarrow C \end{array}$$

8.4.4 无损分解

如果用两个关系模式 $r_1(R_1)$ 和 $r_2(R_2)$ 替代 $r(R)$ 时没有信息损失，则我们称该分解是**无损分解**

不是无损分解的分解称为**有损分解**

R_1 和 R_2 是 R 的无损分解，如果以下函数依赖中至少有一个属于 F^+ ：

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

example:

为举例说明，考虑模式

inst_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)

我们在 8.1.2 节中将其分解为 *instructor* 和 *department* 模式：

instructor (*ID*, *name*, *dept_name*, *salary*)
department (*dept_name*, *building*, *budget*)

考虑这两个模式的交集，即 *dept_name*。我们发现，由于 $dept_name \rightarrow dept_name, building, budget$ ，因此满足无损分解条件。

8.4.5 保持依赖

F 在 R_i 上的**限定**是 F^+ 中所有只包含 R_i 中属性的函数依赖的集合 F_i

令 $F' = F_1 \cup F_2 \cup \dots \cup F_n$ ，若分解具有性质 $F'^+ = F^+$ ，则该分解被称为**保持依赖的分解**

验证方法一：

```

计算  $F^+$  ;
for each  $D$  中的模式  $R_i$  do
    begin
         $F_i := F^+$  在  $R_i$  中的限定;
    end
 $F' := \emptyset$ 
for each 限定  $F_i$  do
    begin
         $F' = F' \cup F_i$ 
    end
计算  $F'^+$  ;
if ( $F'^+ = F^+$ ) then return (true)
else return (false);

```

图 8-10 保持依赖性的验证

验证方法二：

```

result =  $\alpha$ 
repeat
    for each 分解后的  $R_i$ 
         $t = (result \cap R_i)^+ \cap R_i$ 
        result = result  $\cup$   $t$ 
until (result 没有变化)

```

8.5 分解算法

8.5.1 BCNF分解

8.5.1.1 BCNF的判定方法

方法一：（对某些存在 F 中不存在而 F^+ 中的函数依赖情况不适用）

- 为了检查非平凡的函数依赖 $\alpha \rightarrow \beta$ 是否违反 BCNF，计算 α^+ (α 的属性闭包)，并且验证它是否包含 R 中的所有属性，即验证它是否是 R 的超码。
- 检查关系模式 R 是否属于 BCNF，仅须检查给定集合 F 中的函数依赖是否违反 BCNF 就足够了，不用检查 F^+ 中的所有函数依赖。

方法二：

- 对于 R_i 中属性的每个子集 α ，确保 α^+ (F 下 α 的属性闭包) 要么不包含 $R_i - \alpha$ 的任何属性，要么包含 R_i 的所有属性。

即，如果存在这样的函数依赖 $\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$ 在 F^+ 中，说明 R_i 违反 BCNF

8.5.1.2 BCNF分解算法

```

result := {R};
done := false;
计算  $F^+$ ;
while (not done) do
    if (result 中存在模式  $R_i$  不属于 BCNF)
    then begin
        令  $\alpha \rightarrow \beta$  为一个在  $R_i$  上成立的非平凡函数依赖, 满足  $\alpha \rightarrow R_i$  不属于  $F^+$ , 并且  $\alpha \cap \beta = \emptyset$ ;
        result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );
    end
else done := true;

```

图 8-11 BCNF 分解算法

example:

模式:

class (course_id, title, dept_name, credits, sec_id, semester, year, building, room_number, capacity, time_slot_id)

函数依赖:

course_id $\rightarrow$ title, dept_name, credits

building, room_number $\rightarrow$ capacity

course_id, sec_id, semester, year $\rightarrow$ building, room_number, time_slot_id

分解结果:

course(course_id, title, dept_name, credits)

classroom (building, room_number, capacity)

section (course_id, sec_id, semester, year, building, room_number, time_slot_id)

这三个关系模式都属于BCNF

8.5.2 3NF分解

```

令  $F_c$  为  $F$  的正则覆盖;
 $i := 0$ ;
for each  $F_c$  中的函数依赖  $\alpha \rightarrow \beta$ 
     $i := i + 1$ ;
     $R_i := \alpha\beta$ ;
if 模式  $R_j, j=1, 2, \dots, i$  都不包含  $R$  的候选码
then
     $i := i + 1$ ;
     $R_i := R$  的任意候选码;
/* (可选) 移除冗余关系 */
repeat
    if 模式  $R_j$  包含于另一个模式  $R_i$  中
    then
        /* 删除  $R_j$  */
         $R_j := R_i$ ;
         $i := i - 1$ ;
until 不再有可以删除的  $R_j$ 
return( $R_1, R_2, \dots, R_i$ )

```

图 8-12 到 3NF 的保持依赖且无损的分解

example:

关系模式：

cust_banker_branch (customer_id, employee_id, branch_name, type)

函数依赖：

customer_id, employee_id $\rightarrow$ branch_name, type

employee_id $\rightarrow$ branch_name

customer_id, branch_name $\rightarrow$ employee_id

正则覆盖：

customer_id, employee_id $\rightarrow$ type

employee_id $\rightarrow$ branch_name

customer_id, branch_name $\rightarrow$ employee_id

分解结果

(customer_id, employee_id, type)

(customer_id, branch_name, employee_id)

8.5.3 3NF算法的正确性

(好像不重要，不管了)

8.5.4 BCNF和3NF的比较

3NF的优缺点：

优点：总可以在满足无损并保持依赖的前提下得到3NF设计

缺点：可能不得不用空值表示数据项间的某些可能有意义的联系

数据库设计目标：

1. BCNF
2. 无损
3. 保持依赖

8.6 使用多值依赖的分解

令 $r(R)$ 为一关系模式，并令 $\alpha \subseteq R$ 且 $\beta \subseteq R$ ，**多值依赖** $\alpha \twoheadrightarrow \beta$ 在 R 上成立的条件是，在关系 $r(R)$ 的任意合法实例中，对于 r 中任意一对满足 $t_1[\alpha] = t_2[\alpha]$ 的元组对 t_1 和 t_2 ， r 中都存在元组 t_3 和 t_4 ，使得

$$t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha]$$

$$t_3[\beta] = t_1[\beta]$$

$$t_3[R - \beta] = t_2[R - \beta]$$

$$t_4[\beta] = t_2[\beta]$$

$$t_4[R - \beta] = t_1[R - \beta]$$

$\alpha \twoheadrightarrow \beta$ 的表格表示:

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

We say that $Y \twoheadrightarrow Z$ (Y **multidetermines** Z) if and only if for all possible relations $r(R)$

$$\langle y_1, z_1, w_1 \rangle \in r \text{ and } \langle y_1, z_2, w_2 \rangle \in r$$

then

$$\langle y_1, z_1, w_2 \rangle \in r \text{ and } \langle y_1, z_2, w_1 \rangle \in r$$

由多值依赖的定义，我们可以得出以下规则，对于 $\alpha, \beta \subseteq R$:

- 若 $\alpha \rightarrow \beta$ ，则 $\alpha \twoheadrightarrow \beta$ 。换句话说，每一个函数依赖也是一个多值依赖。
- 若 $\alpha \twoheadrightarrow \beta$ ，则 $\alpha \twoheadrightarrow R - \alpha - \beta$ 。

8.8 数据库设计过程

在本章的前面，从 8.3 节开始，我们假定给定关系模式 $r(R)$ ，并对之进行规范化。我们可以采用以下几种方法得到模式 $r(R)$:

1. $r(R)$ 可以由 E-R 图向关系模式集进行转换时生成的。
2. $r(R)$ 可以是一个包含所有有意义的属性的单个关系。然后规范化过程中将 R 分解成一些更小的模式。
3. $r(R)$ 可以是关系的即席设计的结果，然后我们检验它们是否满足一个期望的范式。

8.10 总结

见 P206